Emergency Distress System

An Integrative Programming and Technologies Plan

Davao del Norte State College

New Visayas, Panabo City

In Fulfillment of the

Requirements for the Course

IT312 – Integrative Programming and Technologies

Submitted by:

Madrona, Cristian li G.

UI/UX Designer

Labad, Rodsteven S.

Backend / Flutter Developer

Batawang, Homer J.

Quality Assurance / Tester

Acido, Elishama

Documentation Manager

TITLE: Emergency Distress System

Project Description

The Emergency Distress System is a cross-platform emergency response application developed using Flutter for mobile and web platforms, supported by an API-based backend system. The project is designed to provide individuals and communities with a fast, reliable, and accessible means of requesting emergency assistance during critical situations.

The system allows users to trigger a distress or SOS alert with a single action, automatically transmitting their real-time location, selected emergency type, and user information to the server. These alerts are immediately made available to administrators and responders through a centralized admin dashboard and real-time map interface.

The Emergency Distress System includes features such as live alert tracking, responder assignment, status updates, emergency guidance content, and notification services. Administrators can monitor incoming alerts, manage users and responders, update safety resources, and analyze system performance through reports and dashboards. By integrating real-time communication, geolocation, and secure data handling, the system aims to improve emergency response efficiency, enhance public safety, and support coordinated decision-making during emergencies.

Objectives

The primary goal of the Emergency Distress System is to enhance public safety by providing a reliable digital platform that enables rapid emergency alerting, effective responder coordination, and centralized administrative oversight.

Specific Objectives

1. Client-Side (Mobile and Web Application)

1.1. Enable users to send instant SOS or emergency alerts containing real-time geolocation and emergency type.

1.2. Provide immediate confirmation and status updates to users after an alert is triggered.

1.3. Display emergency resources such as hospitals, police stations, fire stations, and emergency contact numbers.

1.4. Provide step-by-step emergency procedures, safety guidance, and first-aid instructions for various emergency scenarios.

1.5. Allow users to view active alerts, warnings, and system advisories when applicable.

1.6. Support offline access to critical safety guides and emergency instructions during limited connectivity.

1.7. Maintain a simple, intuitive, and responsive user interface optimized for use under stressful situations.

2. Server-Side (Admin Dashboard and API)

2.1. Monitor, validate, and manage incoming distress alerts in real time.

2.2. Display alerts on an interactive map with status indicators, severity levels, and filtering options.

2.3. Manage user accounts, responder profiles, and role-based access control.

2.4. Assign responders to emergencies and track response progress and availability status (available, busy, offline).

2.5. Organize and maintain emergency-related content such as safety guides, resources, and contact information.

2.6. Provide secure authentication, authorization, and audit logging for administrative actions.

3. System Performance and Reliability

3.1. Ensure secure and encrypted communication between the Flutter application and the backend API.

3.2. Maintain accurate and consistent storage of alerts, user data, responder information, and system logs.

3.3. Guarantee reliable delivery of notifications through push notifications, in-app messaging, or SMS where applicable.

3.4. Optimize system performance to support multiple concurrent emergency alerts without failure.

3.5. Implement error handling, data validation, and backup mechanisms to ensure system stability.

4. System Evaluation and Usability

4.1. Conduct usability testing with users, responders, and administrators to evaluate ease of use.

4.2. Collect feedback on interface clarity, navigation flow, and emergency response efficiency.

4.3. Refine system features, workflows, and UI/UX design based on evaluation results.

4.4. Assess the system's effectiveness as a dependable emergency alerting and management platform.

DATA FLOW

The Emergency Distress System follows a structured data flow process. When a user triggers an SOS alert, the Flutter application captures the user's location, selected emergency type, and profile data. This information is securely transmitted to the backend API, where it is stored in the database and immediately forwarded to the admin dashboard. Administrators and responders view the alert on a real-time map, assign responders, update alert status, and send notifications. All actions are logged for monitoring, analytics, and reporting purposes.

UI DESIGNS

The user interface of the Emergency Distress System is designed with simplicity, clarity, and accessibility as priorities. The mobile and web interfaces use intuitive layouts, clear icons, and minimal interaction steps to ensure fast operation during emergencies. The admin dashboard presents alerts, maps, analytics, and management tools in an organized layout to support efficient monitoring and decision-making.

The Emergency Distress System is a comprehensive emergency response solution that integrates real-time alerting, geolocation, responder coordination, and administrative management. Through the use of Flutter and an API-based backend, the system provides a scalable, secure, and user-friendly platform aimed at improving emergency response outcomes and enhancing public safety.